ROS2로 시작하는 로봇 프로그래밍

1장 ~ 4장 독서토론 발표 자료 | 발표자: 배지우

목차

01 ROS의 개념과 구조

- 메타 운영체제로서의 역할
- 기본 통신 아키텍처
- 노드 기반 분산 시스템
- ROS Graph 개념

02 ROS 생태계와 개발 문화

- 글로벌 오픈소스 커뮤니티
- 패키지 생태계
- 개발 정보 찾는 방법
- ROSCon과 ROS Discourse

03 ROS 개발 환경 구축

- Ubuntu 기반 개발 환경
- ament와 colcon 빌드 시스템
- 워크스페이스 구조
- 개발 도구 설정

04 왜 ROS2로 가야하는가?

- ROS1의 한계 극복
- 산업용 플랫폼으로의 진화
- 커뮤니티의 ROS2 전환
- 글로벌 기업의 참여

1~4장의 목적

1장. ROS의 개념과 구조

- 메타 운영체제(Meta OS)로서의 역할
- 기본 통신 아키텍처 이해
- 노드 기반 분산 시스템
- ROS Graph 개념

2장. ROS 생태계와 개발 문화

- 전 세계 개발자가 참여하는 커뮤니티
- 오픈소스 중심의 개발 방식
- 패키지 생태계와 재사용성
- 협업 문화

3장. ROS 개발 환경 구축

- Ubuntu, VSCode, Colcon 등
- 표준 로봇 소프트웨어 개발 환경
- 워크스페이스 구조
- 빌드 시스템 이해

4장. 왜 ROS2로 가야하는가?

- ROS1의 한계 극복
- 산업용(Production-ready) 플랫폼
- DDS 기반 통신
- 실시간 제어와 보안 강화

로봇 소프트웨어 개발의 특수성

센서 시스템	카메라, LiDAR, IMU 등 다양한 센서가 환경 데이터를 수집
제어 시스템	모터, 로봇 팔, 액추에이터를 제어하는 시스템
지능 알고리즘	SLAM, 경로 계획, 물체 인식 등 로봇의 의사결정을 담당
사용자 인터페이스	로봇 상태를 시각화하거나 제어하는 인터페이스
핵심 문제	모듈 간 통신 구조와 데이터 흐름 관리가 핵심

로봇 시스템 아키텍처 문제

프로그램 분리 문제

- 각 기능이 서로 다른 프로그램으로 분리
- 센서, 제어, 알고리즘이 독립적으로 존재
- 데이터 교환 구조 필요
- 시스템 통합의 어려움

독립 실행 구조

- 센서 드라이버가 독립 실행
- 상태 추정 모듈 독립 실행
- 경로 계획 모듈 독립 실행
- 제어 시스템 독립 실행

통신 구현의 복잡성

- 네트워크 통신 구현 필요
- 데이터 직렬화 문제
- 프로세스 관리의 복잡성
- 매번 반복되는 문제 해결

ROS 플랫폼의 등장

- 통신 구조 표준화 제공
- 모듈 간 데이터 교환 지원
- 프로세스 관리 자동화
- 로봇 개발에 집중 가능

ROS란 무엇인가?

정의	Robot Operating System - 로봇을 위한 오픈소스 미들웨어 플랫폼
통신 인프라	노드 간 메시지 기반 통신 구조 (프로세스 사이의 메시지 패싱)
개발 도구	RViz, Gazebo, rqt 등 개발 및 디버깅 도구 제공
로봇 기능 라이브러리	Navigation, SLAM, Manipulation 등 센싱, 인식, 모션 플래닝 기능
글로벌 생태계	전 세계 개발자가 참여하는 패키지 관리 및 오픈소스 커뮤니티

ROS 아키텍처 개념

Node (노드)

- 최소 실행 단위의 독립된 프로세스
- 각 기능이 독립적으로 실행
- 에러 격리와 확장이 용이
- 분산 시스템의 기본 구성 요소

Topic, Service, Action

- 노드 간 메시지 통신 인터페이스
- Topic: 비동기 단방향 메시지 전송
- Service: 동기 요청-응답 방식
- Action: 장시간 작업을 위한 피드백 제공

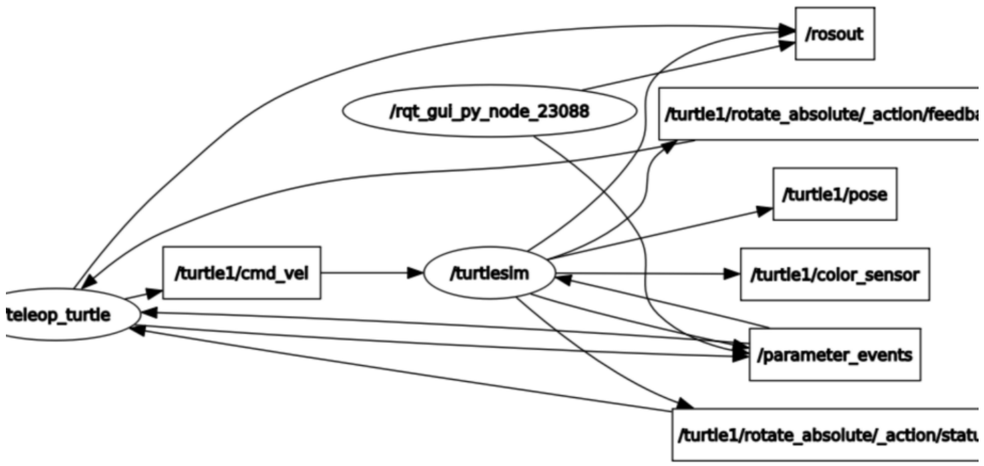
Parameter (파라미터)

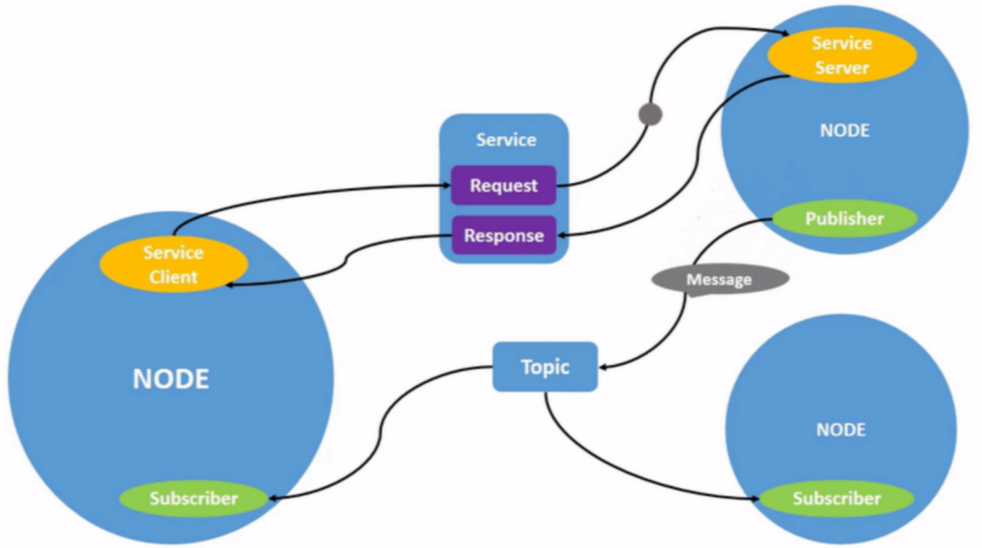
- 노드의 설정값을 관리하는 글로벌 변수
- 런타임 중 동적으로 변경 가능
- 시스템 전체에서 공유되는 설정
- 노드 동작을 유연하게 제어

Package (패키지)

- 실행 가능한 노드와 라이브러리의 묶음 단위
- 코드 재사용성과 배포를 극대화
- 논리적 관리 단위로 기능 분리
- ROS 생태계의 기본 공유 단위

ROS 아키텍처의 핵심 특징





노드 기반 분산 시스템

- 각 기능이 독립적으로 실행되는 구조
- 에러 발생 시 다른 노드에 영향 최소화
- 시스템 확장과 유지보수가 용이
- 모듈화된 개발 방식 지원

메시지 기반 통신 구조

- 표준화된 인터페이스로 데이터 흐름 관리
- 패키지 기반 구조로 코드 재사용성 극대화
- ROS Graph를 통한 시각적 연결 관계 표현
- 명확한 데이터 흐름으로 디버깅 용이

메타
운영체제

ROS는 커널을 가진 전통적인 OS가 아니라 애플리케이션 간 상호작용을 돕는 미들웨어 레이어입니다.
기존 운영체제 위에서 동작하며 하드웨어 추상화와 분산 시스템 관리 기능을 제공합니다.

기존 OS 위에서 동작	- Linux(Ubuntu), Windows, macOS 등 전통적 운영체제 위에 설치 - 운영체제의 기능을 활용하여 로봇 애플리케이션 실행 - 플랫폼 독립적인 개발 환경 제공
하드웨어 추상화	- 복잡한 센서와 액추에이터 통신을 추상화 - 개발자가 하드웨어 세부사항 대신 로직에 집중 - 다양한 하드웨어를 통일된 인터페이스로 제어
분산 시스템 관리	- 여러 컴퓨터와 장치를 하나의 로봇 시스템처럼 연결 - 네트워크를 통한 노드 간 통신 지원 - 대규모 로봇 시스템 구축 가능

ROS의 목적과 핵심 철학

Don't reinvent the wheel	바퀴를 다시 발명하지 마라 - 이미 존재하는 우수한 기능을 재사용하여 핵심 로직에 집중
Time to market 단축	기존 통신, 시각화 프레임워크를 재개발하지 않고 신속하게 애플리케이션 개발
코드 재사용성	기능별로 분리된 수천 개의 패키지를 가져와 쉽게 조립하여 시스템 구성
Apache 2.0 라이선스	지적 재산권 충돌 없이 상용 제품에 마음껏 적용 가능한 상업 친화적 라이선스
글로벌 협업 환경	전 세계 레벨에서 공동 작업이 가능하도록 하는 로보틱스 소프트웨어 개발 환경 구축

대표적인 ROS 패키지 생태계

Navigation2	경로 계획 및 자율 주행을 위한 패키지로 로봇의 자율 이동 기능 제공
MoveIt	로봇 팔 매니퓰레이션 및 모션 플래닝을 위한 패키지로 정밀한 동작 제어 지원
Cartographer	환경 지도 작성 및 위치 추정(SLAM)을 위한 패키지로 로봇의 공간 인식 기능 제공
Gazebo	물리 엔진 기반 3D 로봇 시뮬레이터로 실제 로봇 없이 개발 및 테스트 가능
RViz	강력한 3D 센서 데이터 시각화 도구로 로봇 상태와 센서 정보를 실시간 확인

ROS 개발 역사

연구 플랫폼으로 출발

2007년 Stanford AI Lab에서 ROS 프로젝트 시작

2010년 ROS1 공개

산업용 플랫폼으로 진화

2014년 ROS2 개발 시작

2017년 ROS2 첫 공식 릴리즈

상용 로봇 시스템 플랫폼

2020년 ROS1 마지막 버전 릴리즈

현재 ROS2 중심 생태계로 전환

ROS 배포판 관리 체계

알파벳 순서 릴리즈	거북이 심볼	버전 주기	지원 기간
- 각 릴리즈는 알파벳 순서에 따라 이름이 부여됨 - 예: Dashing, Eloquent, Foxy, Galactic 등 - 체계적인 버전 관리 가능	- ROS는 거북이를 공식 심볼로 사용 - 각 배포판마다 고유한 거북이 로고 존재 - 커뮤니티의 전통적 상징	- 1년에 한 번, 매년 5월에 새 버전 공개 - 정기적인 업데이트 주기 유지 - 예측 가능한 릴리즈 일정	- 기본 지원 기간: 릴리즈 후 2년 - LTS(Long Term Support) 버전: 5년 지원 - 안정적인 장기 운영 가능

ROS 배포판은 알파벳 순서로 명명되며 거북이 심볼을 사용합니다. 매년 5월에 새 버전이 공개되고, 기본 2년 지원(LTS는 5년)을 제공하여 체계적인 버전 관리와 안정적인 운영이 가능합니다.

세계 최대의 로봇 오픈소스 생태계

글로벌 커뮤니티	전 세계 로봇 개발자들이 참여하는 오픈소스 플랫폼의 허브
다양한 참여 주체	학생, 연구자, 로봇 스타트업, 대기업 등 다양한 주체가 협업
ROSCon	ROS 개발자들의 연례 컨퍼런스로 최신 기술과 사례 공유
ROS Discourse	공식 개발 토론 포럼으로 커뮤니티 소통의 중심
GitHub	ROS 소스 코드가 관리되는 공식 저장소

ROS 개발 정보 찾는 방법 (1)

ROS Discourse

- 공식 커뮤니티 포럼
- 새로운 패키지 발표 및 개발 동향 확인
- 워킹 그룹 소식의 허브

ROS Documentation

- ROS2 공식 튜토리얼 제공
- API 문서 및 설치 가이드
- 가장 기본적인 학습 자료

ROS2 디자인 문서

- 개발 설계 중심 문서
- ROS2의 목적과 개발 방향 이해
- 아키텍처 설계 철학 파악

ROSCon 페이지

- 2012년부터의 발표 자료와 영상
- 최신 기술 동향 학습
- 실제 사례 연구 가능

ROS 개발 정보 찾는 방법 (2)

ROS Answers	에러 및 문제 해결을 위한 질의응답 공간
ROS Wiki	ROS1 저장소이나 ROS2의 부족한 문서를 보완하는 참고 자료
GitHub	실제 소스 코드, 패키지, 이슈 트래커가 관리되는 저장소
ROS 배포판 리포지터리	소스코드 이해 및 수정 사항 확인 가능
ROS2 패키지 배포 현황	릴리즈 계획과 패키지 상태를 파악할 수 있는 페이지

ROS2 개발 환경

운영체제

- 주로 Ubuntu 환경 사용
- 각 ROS2 버전별로 지원하는 Ubuntu 버전 확인 필수
- Linux, Windows, macOS 지원

개발 환경 설치 가이드 블로그

- 참고 링크: https://swbee.tistory.com/27
- Ubuntu 버전별 포팅 정보 제공
- 설치 전 확인 권장

Jazzy 공식 설치 문서

- 공식 설치 가이드: https://docs.ros.org/en/jazzy/Installation/Ubuntu-Install-Debs.html
- 단계별 설치 안내

버전 호환성

- ROS2 배포판과 Ubuntu 버전 호환성 확인 중요
- 잘못된 조합 시 설치 및 실행 오류 발생
- 공식 문서 참고 필수

ROS2 빌드 시스템과 워크스페이스

빌드 도구

- ament: CMake 기반 C++ 빌드와 Python setuptools를 모두 지원하는 메타 빌드 시스템
- colcon: 여러 패키지를 한 번에 병렬로 빌드하고 테스트할 수 있는 강력한 범용 CLI 도구

워크스페이스 구조

- robot_ws/ (루트)
- src/: 소스 및 패키지 | build/: 중간 파일 | install/: 빌드 결과 | log/: 로그
- ~/.bashrc에 실행 명령 등록 권장

왜 ROS2로 가야 하는가?

ROS1의 한계 극복	DDS 통신, 분산 아키텍처, 실시간 제어 및 보안 강화로 상용 로봇에 적합
커뮤니티의 강력한 권장	ROS1 마지막 버전 이후 수년 경과, 커뮤니티에서는 ROS2부터 시작하는 것을 적극 권장
배포판 사용률 증가 추세	metrics.ros.org에서 확인 가능한 ROS2 배포판의 지속적인 사용률 증가 및 생태계 확장
산업계의 적극적 관심	ROSCon2019 발표 주제의 50% 이상이 ROS2 관련일 정도로 산업계와 연구계의 높은 관심
글로벌 기업의 참여	ROS2 TSC를 통해 글로벌 기업들이 커뮤니티 서포트 및 ROS2 코어 개발에 적극 공헌

Summary

ROS는 로봇 애플리케이션용 오픈소스 미들웨어로,
노드 기반 분산 구조와 메시지 통신으로 복잡한 시스템을 모듈화해 개발합니다.

ROS는 특정 기업의 독점이 아닌 전세계 개발자 참여의 오픈 생태계로,
수천 패키지로 코드 재사용과 협업을 지원합니다.

ROS2는 ROS1 한계를 보완해 DDS 기반 통신, 탈중앙 아키텍처, 실시간 제어와 보안 강화를 통해 상용 로봇에 적합하게 발전했습니다.

ROS2는 단순 도구가 아니라 현대 로봇 소프트웨어 아키텍처의 표준 플랫폼입니다.